

Univerzitet u Beogradu
Matematički fakultet

Verifikacija softvera

Analiza projekta Circa2020

ProjectAnalysisReport

Autor: Luka Djekic
Broj indeksa: 1021/2025

Beograd, 2026.

Contents

1	Uvod	2
2	Kratak opis projekta	2
3	Korišćene tehnike	2
4	Unit testiranje i LCOV	3
4.1	Izbor dela za testiranje	3
4.2	Dodatni testovi	3
4.3	Coverage metodologija i rezultat	3
5	Cppcheck	4
6	Valgrind Memcheck	4
7	Clazy	5
8	Doxygen i Graphviz	6
9	perf	7
10	clang-format	8
11	CI	8
12	Pregled najvažnijih rezultata	8
13	Zaključak	9

1 Uvod

Ovaj izveštaj opisuje analizu projekta *Circa2020* urađenu u okviru samostalnog projekta na kursu *Verifikacija softvera*. Cilj je bio da se isti projekat posmatra iz više uglova: kroz testiranje i pokrivenost koda, statičku analizu, analizu memorije, Qt-specifičnu analizu, pregled strukture i dokumentacije, profilisanje performansi i proveru formatiranja.

Originalni projekat je dostupan na adresi:

<https://gitlab.com/matf-bg-ac-rs/course-rs/projects-2020-2021/20-circa2020>

Originalni izvorni kod submodulea nije menjan. Dodatni testovi, skripte i rezultati nalaze se izvan njega, u roditeljskom repozitorijumu.

2 Kratak opis projekta

Circa2020 je C++/Qt aplikacija za kreiranje i simulaciju digitalnih logičkih kola. Projekat ima logički sloj sa komponentama i vezama između njih, kao i Qt grafički sloj preko kog korisnik postavlja i povezuje komponente.

Klasa `Component` predstavlja jednu od centralnih apstrakcija logičkog dela. Iz nje su izvedene različite komponente, među kojima su `Adder`, `Clock`, `Comparator`, `Gate`, `LightBulb`, `Pin`, `Splitter`, `Subtractor` i `Wire`. Klasa `Gate` dalje predstavlja osnovu za konkretna logička kola kao što su `AndGate`, `NandGate`, `NorGate`, `NotGate`, `OrGate` i `XorGate`.

Grafički deo prati sličnu ideju kroz hijerarhiju zasnovanu na klasi `ComponentGraphicsItem`, koja povezuje logičke komponente sa njihovim prikazom na Qt sceni.

3 Korišćene tehnike

U analizi su korišćene sledeće tehnike i alati:

1. unit testovi uz LCOV analizu pokrivenosti,
2. Cppcheck,
3. Valgrind Memcheck,
4. Clazy,
5. Doxygen uz Graphviz,
6. perf,
7. clang-format.

Za svaki finalno korišćeni alat postoji skripta za reprodukciju analize i odgovarajući rezultat u repozitorijumu.

4 Unit testiranje i LCOV

4.1 Izbor dela za testiranje

Pre pisanja dodatnih testova pregledana je postojeća pokrivenost testovima. Aritmetički deo biblioteke bio je slabo pokriven, pa je za dodatne testove izabrana klasa **Adder**. Ideja je bila da se testiranje usmeri na deo koji postojeći testovi nisu dovoljno obuhvatali, umesto da se ponavljaju već dobro testirane komponente.

Dodatni testovi se nalaze u direktorijumu `unit_tests/` i koriste originalnu biblioteku bez menjanja submodula.

4.2 Dodatni testovi

Napisano je 6 test slučajeva sa ukupno 33 assertions. Testovi proveravaju:

- početno stanje i broj ulaza i izlaza,
- promenu broja ulaza,
- odbijanje nevalidnih dimenzija,
- propagaciju stanja `ERROR` i `UNDEFINED`,
- sabiranje pozitivnih i negativnih vrednosti,
- ponašanje komponente nakon promene veličine.

Adder koristi sign-magnitude reprezentaciju. Podrazumevano ima 6 ulaza, po tri bita za svaki operand, i 4 izlaza. Za aktiviranje logike testovi koriste javni interfejs i `notifyStateChanged()`, umesto direktnog pristupa zaštićenoj metodi `update()`.

Kompletna analiza se pokreće skriptom:

```
./unit_tests/run_tests.sh
```

4.3 Coverage metodologija i rezultat

Finalna LCOV procedura radi u nekoliko koraka: prvo se generiše zero baseline, zatim se pokreću testovi i prikupljaju runtime podaci, nakon čega se ta dva skupa podataka spajaju. Na kraju se rezultat filtrira tako da obuhvati samo produkcionu `20-circa2020/lib/` kod i iz njega se generiše HTML izveštaj.

Ovakav postupak je važan zato što i potpuno neizvršeni produkcionu fajlovi ostaju u ukupnom rezultatu sa pokrivenošću od 0%.

Finalni coverage iznosi:

Table 1: Coverage dodatnih unit testova

Metrika	Pokriveno	Rezultat
Linije	131 / 2502	5.2%
Funkcije	26 / 416	6.2%

Ukupan procenat nije visok jer su dodatni testovi namerno fokusirani na **Adder**, dok imenilac obuhvata celu produkcionu biblioteku. HTML izveštaj se generiše na lokaciji:

`unit_tests/coverage/html/index.html`

5 Cppcheck

Cppcheck 2.7 je korišćen za statičku analizu produkcionog `lib/` dela projekta. Pokretanje je automatizovano skriptom:

```
./cppcheck/run_cppcheck.sh
```

Pronađena su tri glavna upozorenja.

Prvo upozorenje se odnosi na pozivanje virtuelne funkcije `removeAll()` iz destruktora klase `CircuitBoard`. Ovo nije potvrđen funkcionalni bug, već upozorenje na osetljiv C++ obrazac i dizajn klase.

Druga dva upozorenja odnose se na funkcije:

```
Converter::makeSingleNode
Converter::makeMultiInputNode
```

One primaju `std::set<Component *> visited` po vrednosti. Time može nastati nepotrebno kopiranje, pa je ovaj nalaz pre svega performance/code-quality problem.

Kompletna rezultat se čuva u `cppcheck/results.txt`.

6 Valgrind Memcheck

Valgrind Memcheck je korišćen nad stvarnom Qt GUI aplikacijom, kako bi analiza obuhvatila realan tok korišćenja, a ne samo test executable.

Skripta:

```
./valgrind/run_valgrind.sh
```

najpre izgradi aplikaciju, a zatim je pokreće pod Memcheck-om.

Tokom analize korišćen je sledeći funkcionalni scenario:

1. pokretanje aplikacije,
2. dodavanje dve `Pin` komponente,
3. dodavanje `AND` kola i `LightBulb` komponente,
4. povezivanje pinova sa `AND` kolom i `AND` izlaza sa sijalicom,
5. promena stanja ulaza i provera ponašanja,
6. normalno zatvaranje aplikacije.

Za ovaj scenario dobijen je sažetak:

```
definitely lost:    256 B in 1 block
indirectly lost:   32 B in 1 block
possibly lost:     1920 B in 5 blocks
still reachable: 2536638 B
ERROR SUMMARY: 43 errors from 13 contexts
```

Sam broj prijavljenih grešaka nije dovoljan da bi se zaključilo da problem potiče iz projekta. Pregled stack trace-ova pokazao je da se analizirani konteksti pojavljuju u eksternim ili sistemskim bibliotekama, kao što su `Qt`, `GTK/GLib`, `GLX`, dinamički loader i `AppProtection` biblioteka.

U pregledanim kontekstima nisu pronađeni frejmovi koji bi problem direktno povezali sa `20-circa2020/lib/` ili `20-circa2020/app/` izvornim kodom. Zato je zaključak da `Memcheck` jeste prijavio greške tokom izvršavanja, ali da u testiranom scenariju nije pronađena greška koja se može direktno pripisati `Circa2020` kodu.

Rezultat se čuva u `valgrind/results.txt`.

7 Clazy

`Clazy` 1.11 je korišćen kao `Qt`-specifični statički analizator. Za razliku od `Cppcheck`-a, `Clazy` poznaje tipične `Qt` obrasce, pa može da prijavi probleme vezane za implicit sharing, detach operacije, signal/slot konvencije i `Qt` tipove.

Za finalnu analizu generisan je `compile_commands.json`, aplikacija je izgrađena kako bi bili dostupni `Qt`-generated fajlovi, a zatim je pokrenuta `level1` analiza nad `lib/` i `app/` delovima projekta.

Skripta za reprodukciju je:

```
./clazy/run_clazy.sh
```

Pošto isti warning može da se pojavi više puta kroz različite translation unit-e, warning linije su deduplikovane pre konačnog brojanja.

Dobijeno je 26 jedinstvenih upozorenja:

Table 2: Clazy upozorenja nakon deduplikacije

Kategorija	Broj
connect-by-name	8
detaching-temporary	8
range-loop-detach	2
range-loop-reference	5
unused-non-trivial-variable	2
qcolor-from-literal	1

Jedan primer `detaching-temporary` upozorenja nalazi se u `scene.cpp`:

```
if (!selectedItems().empty())
    selectedItems().first()->setSelected(false);
```

`selectedItems()` vraća privremeni Qt kontejner, pa ovakav pristup može dovesti do nepotrebnog detach/copy ponašanja.

Drugi čest tip upozorenja je `range-loop-reference`. Na primer:

```
for (auto formula : expressionFormulae)
    os << formula << std::endl;
```

Pošto se objekat kopira pri svakoj iteraciji, efikasniji oblik bio bi korišćenje `const` reference:

```
for (const auto& formula : expressionFormulae)
```

Upozorenja `connect-by-name` odnose se na Qt auto-connect konvenciju, koja zavisi od imena UI objekata i zato može biti osetljiva na kasnija preimenovanja.

Kompletnan output i deduplikovana lista nalaze se u:

```
clazy/results_level1.txt
clazy/unique_warnings.txt
```

8 Doxygen i Graphviz

Doxygen 1.9.1 i Graphviz 2.43.0 korišćeni su kao dodatna tehnika za pregled strukture projekta i postojeće dokumentacije.

Pokretanje:

```
./doxygen/run_doxygen.sh
```

Generisana dokumentacija uključuje class, inheritance, collaboration, include i included-by grafove. Oni daju pregled odnosa između logičkih komponenti i grafičkog sloja projekta.

Doxygen je prijavio jedno konkretno upozorenje:

```
componentfactory.cpp:310: warning:
argument 'new_copy_callback' of command @param is not found
in the argument list of ComponentFactory::deepCopy(...)
```

Komentar dokumentuje parametar `new_copy_callback`, dok trenutni potpis funkcije koristi `ShouldCopyPredicate should_copy`. To nije funkcionalni bug, ali jeste jasna neusklađenost dokumentacije i implementacije.

Upozorenje se čuva u `doxygen/warnings.txt`, dok se HTML dokumentacija generiše u `doxygen/output/html/`.

9 perf

Linux `perf` je korišćen za profilisanje performansi postojećeg test suite-a.

Analiza se pokreće skriptom:

```
./perf/run_perf.sh
```

Skripta pravi čist `Release` build bez coverage instrumentacije, pokreće testove, zatim izvršava `perf stat` i `perf record`. Pre profilisanja proverava se da executable ne sadrži `gcov` simbole.

Konačni `perf stat` rezultat, kroz pet ponavljanja, prikazan je u tabeli:

Table 3: Konačni perf stat rezultat	
Metrika	Vrednost
Task clock	5.41 ms
Cycles	12,124,859
Instructions	10,151,895
IPC	0.84
Frontend stalled cycles	5.27%
Backend stalled cycles	18.39%
Branch misses	6.40%
Elapsed time	≈ 7.20 ms

Backend stall je veći od frontend stall-a, dok IPC iznosi 0.84. Ove vrednosti opisuju konkretan test workload i ne predstavljaju same po sebi funkcionalni problem.

Pošto jedan prolaz test suite-a traje svega nekoliko milisekundi, `perf record` koristi 100 pokretanja kako bi prikupio dovoljan broj uzoraka. U profilu se među projektno relevantnim funkcijama pojavljuju:

- `Component::notifyStateChanged(...)`,
- `Wire::update()`,
- `ComponentFactory::deepCopy(...)`,

- `Connection::setSignal(...)`,
- `Component::setInput(...)`,
- `Component::propagateInputs()`.

Rezultati se čuvaju u `perf/stat.txt`, `perf/report.txt`, `perf/project_report.txt` i `perf/test_output.txt`.

10 clang-format

clang-format 14 korišćen je za proveru usklađenosti izvornog koda sa postojećom konfiguracijom projekta:

```
20-circa2020/.clang-format
```

Novi stil nije definisan; proveravana su pravila koja su već postavili autori projekta.

Pokretanje:

```
./clang-format/run_clang_format.sh
```

Skripta koristi `-dry-run -Werror`, pa ne menja izvorni kod.

Provereno je ukupno 183 `.cpp`, `.hpp` i `.h` fajla iz `lib/` i `app/` direktorijuma.

Rezultat je:

```
Checked files: 183  
Non-compliant files: 0
```

Dakle, svi analizirani fajlovi usklađeni su sa postojećom clang-format konfiguracijom.

11 CI

Repozitorijum sadrži zvanične MATF Software Verification GitHub Actions workflow-e u direktorijumu `.github/workflows/`.

`gate.yml` pokreće zvanični `VS-project-ci` gate na `push` i `pull_request` događaje, dok `tickets.yml` proverava da li postoje otvoreni GitHub issue-i.

12 Pregled najvažnijih rezultata

Tehnika	Najvažniji rezultat
Unit testovi + LCOV Cppcheck	Dodatno testiran Adder ; 6 test slučajeva, 33 assertions; coverage cele produkcione biblioteke 5.2% linija i 6.2% funkcija. Jedno upozorenje na dizajn zbog virtuelnog poziva iz destruktora i dva performance upozorenja zbog prosleđivanja <code>std::set</code> po vrednosti.
Valgrind Mem-check	Prijavljeni su memory/error konteksti, ali pregledani stack trace-ovi nisu pokazali problem koji se može direktno pripisati Circa2020 kodu.
Clazy	26 jedinstvenih Qt-specifičnih upozorenja, najviše vezanih za detach, range-for kopiranje i auto-connect konvenciju.
Doxygen	Generisan pregled strukture projekta i pronađena neusklađenost između <code>@param</code> dokumentacije i trenutnog potpisa funkcije.
perf	Dobijen profil test workload-a; IPC 0.84, backend stall 18.39% i branch miss rate 6.40%.
clang-format	Svih 183 analiziranih C/C++ fajlova usklađeno je sa postojećim stilom projekta.

13 Zaključak

Analiza projekta *Circa2020* pokazala je da različiti alati daju različite i međusobno dopunjujuće informacije o kvalitetu sistema.

Dodatni unit testovi proširili su proveru aritmetičkog dela projekta, dok je LCOV dao realnu sliku pokrivenosti kada se u rezultat uključi cela produkciona biblioteka. Cppcheck je ukazao na nekoliko opštih C++ design i performance problema, dok je Clazy pronašao Qt-specifične obrasce koji nisu cilj opšteg statičkog analizatora.

Valgrind Memcheck nije pokazao grešku koja se u korišćenom scenariju može direktno pripisati Circa2020 kodu. Doxygen je dao koristan pregled strukture projekta i pronašao jednu konkretnu neusklađenost dokumentacije. Perf je dao osnovni profil ponašanja postojećeg test workload-a, a clang-format je potvrdio da je analizirani kod usklađen sa stilom koji projekat već definiše.

Originalni *Circa2020* submodule nije menjan tokom analize. Svi dodatni testovi, skripte i rezultati organizovani su izvan njega, tako da se analiza može reprodukovati bez menjanja izvornog projekta.